DO

Document Organizer

COPY-ONLY DOCUMENT RE-ORGANIZER

INSTALLATION AND ADMINISTRATION GUIDE

Installation and Administration Guide

Installing, configuring, connecting to Azure Blob Storage and Amazon S3, hosting and managing users

Document ID DO-IG-02 · **Version** 2.1 · **Date** 22-Oct-2026
Prepared by Yasin Jariwala
Audience Administrators, cloud and infrastructure teams

Document control

Document	Installation and Administration Guide
Document ID	DO-IG-02
Version	2.1
Date	22-Oct-2026
Prepared by	Yasin Jariwala
Product	Document Organizer v2.1
Audience	Administrators, cloud and infrastructure teams
Classification	Public
Source	github.com/yasin679/Document_Organizer · docs/installation-and-administration.md

Version history

Version	Date	Author	Changes
1.0	06-Oct-2026	Yasin Jariwala	First release
2.0	13-Oct-2026	Yasin Jariwala	General-purpose release: Amazon S3 and S3-compatible storage, copy between providers, container hosting; tenant ID field removed
2.1	22-Oct-2026	Yasin Jariwala	Several mapping files per job; File name cells listing several files split into one entry per file; NULL placeholders treated as blank; repeats of missing records reported as MISSING

Related documents

ID	Document
DO-UG-01	User Guide
DO-QS-03	Quick Start
DO-FAQ-04	FAQ and Troubleshooting
DO-TD-05	Technical Design
DO-UAT-06	UAT Test Cases
DO-RN-07	Release Notes v2.1

Contents

1 About this guide 3
1.1 What is in the repository 3
2 Requirements 4
2.1 Machine 4
2.2 Azure Blob Storage 4
2.3 Amazon S3 and S3-compatible storage 4
3 Installation 6
3.1 Windows 6
3.2 Linux and macOS 6
3.3 Docker 7
3.4 The admin password 7
4 Configuration 8
4.1 Application settings 8
4.2 Storage connections 8
4.3 Azure Blob Storage connections 9
4.4 Amazon S3 and S3-compatible connections 10
4.5 Restricting local folders 11
5 Administration in the application 13
5.1 In-app settings 13
5.2 Users and roles 13
6 Testing with emulators 15
7 Hosting for a team 16
8 Backup, retention and logs 17
9 Upgrading 18
10 Running the automated tests 19
11 Go-live checklist 20

1 About this guide

This guide explains how to install, configure, secure and run Document Organizer. It is written for the administrator who sets the application up on a laptop, a server or a container platform, connects it to Azure Blob Storage and Amazon S3, and manages users.

For day-to-day use of the screens, see the User Guide (DO-UG-01). For the design and data model, see the Technical Design (DO-TD-05).

> **Before you use real documents**
>
> Version 2.0 is tested automatically on Linux with .NET 8 against the Azurite (Azure) and moto (S3) emulators, including an S3 emulator that checks every request signature. Before running a job on real documents, complete the go-live checklist in section 11 on your own machine, against a test container or bucket in your own cloud account.

1.1 What is in the repository

Part	Where it lives	Notes
Source code	`src/DocumentOrganizer/`	.NET 8 web application
Configuration	`src/DocumentOrganizer/appsettings.json`	Copied next to the application when it is built
Start scripts	`start.cmd` (Windows), `start.sh` (Linux, macOS)	Run the prebuilt `app` folder if present, otherwise build and run the source
Container files	`Dockerfile`, `docker-compose.yml`, `.env.example`	Container image, and a local trial with Azure and S3 emulators
Documentation	`docs/`	These guides, as Markdown and PDF
Tests	`tests/e2e.mjs`	End-to-end test script (Node 18 or later)
Sample	`samples/sample-mapping.csv`	Example mapping file
Data folder	`data/` next to the application (created at first start)	SQLite database, uploaded mapping files, demo data. Not in Git.

The application has no third-party packages and needs no database server. SQLite is used through the operating system's library: `winsqlite3.dll` on Windows 10 and 11, `libsqlite3` on Linux and macOS. Azure Blob Storage and Amazon S3 are called through their REST APIs directly, so no cloud SDK is installed.

2 Requirements

2.1 Machine

Item	Requirement
Operating system	Windows 10 or 11 (64-bit) or Windows Server 2019 or later; any Linux distribution supported by .NET 8; macOS 12 or later; or any platform that runs Linux containers
.NET	.NET 8 SDK to build from source, or the ASP.NET Core 8 Runtime to run a prebuilt `app` folder. Not needed with Docker.
SQLite	Built into Windows and macOS. On Linux, the `libsqlite3-0` (Debian/Ubuntu) or `sqlite-libs` (RHEL/Fedora) package. Included in the Docker image.
Memory	4 GB minimum, 8 GB recommended for mapping files above 100,000 rows
Disk	About 100 MB for the application, plus the database (roughly 1 KB per file in a job)
Browser	Current Microsoft Edge, Google Chrome, Firefox or Safari
Network	HTTPS (port 443) to `*.blob.core.windows.net` for Azure jobs and to `s3.<region>.amazonaws.com` (or your S3-compatible endpoint) for S3 jobs

2.2 Azure Blob Storage

Item	Requirement
Storage account	The account that holds the documents
Source access	Read and list on the source container (role **Storage Blob Data Reader**, or a SAS with `r` and `l`)
Destination access	Write on the destination container (role **Storage Blob Data Contributor**, or a SAS with `r`, `l`, `w` and `c`)
Azure CLI	Needed only for the `AzureCli` sign-in method on a laptop

2.3 Amazon S3 and S3-compatible storage

Item	Requirement
Account	An AWS account, or an S3-compatible service (MinIO, Wasabi, Cloudflare R2 and others)
Source access	`s3:ListBucket` and `s3:GetObject` on the source bucket
Destination access	`s3:ListBucket`, `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject` and `s3:AbortMultipartUpload` on the destination bucket; `s3:CreateBucket` only if the application should create a missing bucket
Credentials	An access key, or an IAM role when hosted on AWS (EC2, ECS, EKS)

A ready-to-use IAM policy is in section 4.4.

Same account means faster jobs

When source and destination are in the same Azure storage account, or use the same S3 connection, the application uses server-side copy. Documents never pass through the application server. Across providers (for example Azure to S3) the application streams each file; files over 8 MiB go to S3 as multipart uploads.

3 Installation

Choose one of three ways to run the application.

Option	Best for	Section
Windows laptop or server	One administrator or a small team on Windows	3.1
Linux or macOS	Servers and developer machines	3.2
Docker	Hosting, and a quick trial with both storage emulators	3.3

3.1 Windows

1. Install the .NET 8 SDK from https://dotnet.microsoft.com/download/dotnet/8.0 (Windows x64). Open a new Command Prompt and run `dotnet --version`. It should show a version starting with `8.`.
2. Get the code: `git clone https://github.com/yasin679/Document_Organizer.git`, or download the ZIP from GitHub and unzip it, for example to `C:\Apps\DocumentOrganizer`.
3. Do not run it from a network share or a synced OneDrive folder. The database must be on a local disk.
4. Double-click `start.cmd`.
5. When asked, type a password for the built-in `admin` user and press Enter. It must be at least 8 characters. Pressing Enter without typing keeps the current password.
6. The script builds the application the first time, then opens http://localhost:5080. Keep the console window open while you use the application. Closing it stops the application.
7. Sign in as **admin** with the password you typed.
8. Go to **Settings > My password** and change it to a password of at least 10 characters with three of: upper-case, lower-case, digits and symbols.

To build once and run the prebuilt copy afterwards (faster start, and the only option on a machine that has just the runtime):

```
dotnet publish src\DocumentOrganizer -c Release -o app
```

`start.cmd` then runs `app\DocumentOrganizer.dll`, and the data folder is `app\data`.

3.2 Linux and macOS

1. Install the .NET 8 SDK (https://learn.microsoft.com/dotnet/core/install/linux, or `brew install dotnet@8` on macOS) and, on Linux, the SQLite library (`sudo apt install libsqlite3-0` on Debian/Ubuntu).
2. Clone the repository and start:

```
git clone https://github.com/yasin679/Document_Organizer.git
cd Document_Organizer
DOCORG_ADMIN_PASSWORD='Choose-A-Strong-Pass-1' ./start.sh
```

3. Open http://localhost:5080 and sign in as **admin**.

To run it as a service, publish it (`dotnet publish src/DocumentOrganizer -c Release -o /opt/document-organizer`) and create a systemd unit that runs `dotnet /opt/document-organizer/DocumentOrganizer.dll`

with `WorkingDirectory=/opt/document-organizer`, under a dedicated user that owns the `data` folder.

3.3 Docker

The repository includes a `Dockerfile` and a `docker-compose.yml` that also starts **Azurite** (Azure Blob emulator) and **MinIO** (S3 emulator), so every storage type can be tried without a cloud account.

```
cp .env.example .env        # then set DOCORG_ADMIN_PASSWORD in .env
docker compose up --build
```

Open http://localhost:8080 and sign in as **admin**. The MinIO console is at http://localhost:9001.

To run only the application image:

```
docker build -t document-organizer .
docker run -d -p 8080:8080 -v docorg-data:/app/data \
  -e DOCORG_ADMIN_PASSWORD='Choose-A-Strong-Pass-1' \
  -e DocumentOrganizer__EnableDemoData=false \
  document-organizer
```

The image listens on port 8080, runs as a non-root user and keeps its database in the `/app/data` volume. Configure everything with environment variables (section 4).

3.4 The admin password

If you do not set a password

On the very first start only, if `DOCORG_ADMIN_PASSWORD` is not set, a temporary password is printed in the console (or the container log). Copy it from there. On later starts the existing password is kept.

Forgotten admin password

Stop the application and start it again with `DOCORG_ADMIN_PASSWORD` set to a new password (on Windows, `start.cmd` asks for it). The password is applied at start-up, the `admin` account is re-enabled, the admin's existing sessions are signed out, and the change is written to the audit log. It is applied only when the value is new: restarting with the same value changes nothing, so a password the admin later changes in the application is not undone. Remove the variable afterwards on shared servers.

4 Configuration

Settings are read from `appsettings.json` next to the application, then from `appsettings.Production.json` if present, then from environment variables (which win). Restart the application after changing them.

> **Keep secrets out of Git**
>
> Put account keys, SAS tokens and S3 secret keys in environment variables or in `appsettings.Production.json`. That file is listed in `.gitignore`, so it is never committed. Secrets are never stored in the job database, shown on screen or written to logs.

Environment variables use a double underscore for each level, for example `DocumentOrganizer__SessionTimeoutMinutes=20`.

4.1 Application settings

Setting	Default	Meaning
`Urls`	`http://localhost:5080`	Listen address. The Docker image sets `http://+:8080`. Keep it on localhost unless the application sits behind HTTPS.
`DocumentOrganizer:DataPath`	`data`	Folder for the database, uploaded mapping files and demo data
`DocumentOrganizer:SessionTimeoutMinutes`	`30`	Sign-out after this many minutes of inactivity
`DocumentOrganizer:AllowedLocalRoots`	`[]` (any)	Folders that local jobs may use. Set this on any shared machine.
`DocumentOrganizer:EnableDemoData`	`true`	Shows **Load demo data**. Set to `false` in production.
`DocumentOrganizer:BehindReverseProxy`	`false`	Trust `X-Forwarded-For` and `X-Forwarded-Proto` from a reverse proxy or load balancer (section 7)
`DocumentOrganizer:StorageConnections`	`Azurite`, `LocalS3`	Named cloud connections (sections 4.2 to 4.4). The two defaults point at local emulators; remove them in production.

4.2 Storage connections

Each connection has a name that operators pick from a list on the New job page, and a `Provider` of `AzureBlob` or `S3`. If `Provider` is left out it is inferred: S3 settings (`Region`, `ServiceUrl` or `AccessKeyId`) mean S3, anything else Azure.

Setting	Applies to	Meaning
`Provider`	Both	`AzureBlob` or `S3`
`Description`	Both	Shown to administrators under Settings
`UseForDemo`	Both	When `true`, **Load demo data** also uploads the sample files here. For local emulators only.

4.3 Azure Blob Storage connections

Auth method	Use it when	Settings
`AzureCli`	Running on a laptop with your own Azure sign-in	`AccountUrl`, `Auth`
`ManagedIdentity`	Running on an Azure VM, App Service or Container Apps	`AccountUrl`, `Auth`, optional `ManagedIdentityClientId`
`Sas`	Someone gives you a SAS token	`AccountUrl`, `Auth`, `SasToken`
Connection string	Account key or a full connection string	`ConnectionString`

```
"StorageConnections": {
  "Prod-Azure": {
    "Provider": "AzureBlob",
    "AccountUrl": "https://mystorageaccount.blob.core.windows.net",
    "Auth": "AzureCli",
    "Description": "Production documents (my Azure sign-in)"
  },
  "Client-SAS": {
    "Provider": "AzureBlob",
    "AccountUrl": "https://clientaccount.blob.core.windows.net",
    "Auth": "Sas"
  }
}
```

Set the SAS token as an environment variable rather than in the file:

```
set DocumentOrganizer__StorageConnections__Client-SAS__SasToken=sv=...&sig=...
```

Give the SAS token the shortest practical expiry and only the permissions listed in section 2.2.

Using Azure CLI sign-in on a laptop

1. Install the Azure CLI from https://aka.ms/installazurecli.
2. Run `az login` and sign in with your organization account.
3. If you have several subscriptions, run `az account set --subscription "<name>"`.
4. Ask the Azure administrator to give your account **Storage Blob Data Reader** on the source container and **Storage Blob Data Contributor** on the destination container.

5. Start Document Organizer as the same operating-system user. It requests a storage token through `az account get-access-token` and refreshes it automatically.

> **Role assignment delay**
>
> Azure role assignments can take up to 10 minutes to take effect. If **Check access** says `AuthorizationPermissionMismatch` straight after a role was granted, wait and try again.

4.4 Amazon S3 and S3-compatible connections

Setting	Meaning
`Region`	The bucket's region, for example `ap-south-1`. Defaults to `AWS_REGION` or `AWS_DEFAULT_REGION`, then `us-east-1`. If it is wrong, **Check access** names the right one.
`ServiceUrl`	Endpoint of an S3-compatible service, for example `https://minio.example.com`. Leave it out for AWS.
`ForcePathStyle`	Use `endpoint/bucket/key` addresses. Default `true` when `ServiceUrl` is set, `false` for AWS.
`AccessKeyId`, `SecretAccessKey`, `SessionToken`	An access key (put the secret in an environment variable). `SessionToken` only for temporary keys.
`Auth`	`AccessKey` (default when `AccessKeyId` is set) or `Default` (otherwise). You can also force one source: `Environment`, `Profile`, `WebIdentity`, `Container` or `InstanceProfile`.
`Profile`	Profile name in `~/.aws/credentials`. Default `AWS_PROFILE`, then `default`.

With `Auth: Default` the application looks for credentials in the standard AWS order:

1. Environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` (and `AWS_SESSION_TOKEN`).
2. Web identity, used by EKS service accounts (IRSA): `AWS_WEB_IDENTITY_TOKEN_FILE` and `AWS_ROLE_ARN`.
3. Static keys in the shared credentials file `~/.aws/credentials`.
4. The container role on ECS or EKS Pod Identity.
5. The EC2 instance role, through the instance metadata service (IMDSv2).

```
"StorageConnections": {
  "Prod-S3": {
    "Provider": "S3",
    "Region": "ap-south-1",
    "Description": "Production bucket (IAM role of this server)"
  },
  "Client-S3": {
    "Provider": "S3",
    "Region": "eu-west-1",
    "AccessKeyId": "AKIA..."
  },
  "MinIO": {
    "Provider": "S3",
    "ServiceUrl": "https://minio.example.com",
    "AccessKeyId": "docorg"
  }
}
```

```
export DocumentOrganizer__StorageConnections__Client-S3__SecretAccessKey='...'
export DocumentOrganizer__StorageConnections__MinIO__SecretAccessKey='...'
```

Least-privilege IAM policy (replace the bucket names):

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "ReadSource", "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::source-bucket", "arn:aws:s3:::source-bucket/*"] },
    { "Sid": "WriteDestination", "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject", "s3:PutObject", "s3:DeleteObject",
"s3:AbortMultipartUpload"],
      "Resource": ["arn:aws:s3:::destination-bucket", "arn:aws:s3:::destination-bucket/*"] }
  ]
}
```

- `s3:ListBucket` on the destination is needed so that the check "does this file already exist?" returns *not found* instead of *access denied*.
- `s3:DeleteObject` is used only to remove the application's own small test file when you click **Check access**. The application never deletes documents.
- Add `s3:CreateBucket` only if you want the application to create a missing destination bucket.
- Server-side copy (`CopyObject`) needs one connection that can read the source and write the destination.

4.5 Restricting local folders

On a shared machine, list the folders that local jobs may read from and write to:

```
"AllowedLocalRoots": [ "D:\\HRDocs", "\\\\fileserver\\hr-exports" ]
```

or on Linux `"AllowedLocalRoots": [ "/srv/hr-exports" ]`. Any path outside these roots is refused when the job is created. A destination folder inside the source folder (or the other way round) is always refused.

5 Administration in the application

5.1 In-app settings

Administrators change these under **Settings**. Every change is written to the audit log.

Setting	Default	Meaning
Azure Blob Storage	On	Allows Azure sources and destinations
Amazon S3	On	Allows S3 sources and destinations
Local or network folders	On	Allows folder sources and destinations
Allow overwrite on duplicate	Off	Makes the Overwrite duplicate rule available to operators
Maker-checker (second approver)	Off	A person who did not prepare a job must approve it; exclusions and restores after approval need a second person too. Turn it on when audit requires four-eyes control; you need at least two operators or administrators.
Parallel copies	8	Files copied at the same time (1 to 64). Raise it to 16 to 32 for SSD, NVMe, network shares and cloud storage; lower it to 2 to 4 for a single external hard disk, and if storage reports throttling (HTTP 503 or `SlowDown`).
Keep jobs and reports	365 days	Older jobs, their items and mapping files are removed at start-up
Keep audit log	7 years	Audit events are kept at least this long (minimum 365 days)

5.2 Users and roles

Role	Can do
Viewer	View dashboard, jobs, reports and audit log. Download reports.
Operator	Everything a Viewer can, plus create jobs, upload mapping files, scan, resolve, approve, pause, resume, retry
Administrator	Everything an Operator can, plus settings, users and deleting any job

To add a user:

1. Go to **Settings > Users and roles**.
2. Enter a user name (3 to 64 letters, digits, dot, dash, underscore or @), a display name, a role and an initial password (at least 10 characters with three of: upper-case, lower-case, digits, symbols).
3. Click **Add user**. Send the initial password to the user through a secure channel.
4. The user signs in and changes it under **Settings > My password**.

To remove access, click **Disable**. The user's history stays in the audit log. To reset a forgotten password, click **Reset password** and type a new password for the user. Sign-in blocked after wrong passwords unblocks by itself after 10 minutes. Resetting a password signs that user out everywhere.

Sign-in protection

Passwords are stored as PBKDF2-SHA256 hashes with 210,000 iterations. After 5 wrong passwords from one computer, sign-in for that user from that computer is blocked for 10 minutes (others are not affected). Signing out, changing or resetting a password ends the sessions on the server immediately.

6 Testing with emulators

Two free emulators let you try cloud jobs without touching a real account. With Docker, `docker compose up` starts both (section 3.3). Without Docker:

Azurite (Azure Blob)

1. Install Node.js LTS from https://nodejs.org.
2. In a terminal (on Windows, Command Prompt rather than PowerShell), run:

```
npm install -g azurite
azurite-blob --location ./.azurite --silent
```

MinIO or moto (S3)

Run MinIO (https://min.io) on port 9000 with user and password `minioadmin`, or, with Python installed:

```
pip install "moto[server]"
moto_server -p 9000
```

Then:

1. Leave the emulator windows open and start Document Organizer.
2. On the dashboard click **Load demo data**. Because the emulators are running, the demo files are also uploaded to the `Azurite` and `LocalS3` connections.
3. On the **New job** page click **Fill in Azurite demo containers** or **Fill in LocalS3 demo buckets**.

> **PowerShell error "running scripts is disabled"**
>
> Use `npm.cmd` instead of `npm`, or run the commands from Command Prompt.

7 Hosting for a team

For use by more than one person, host the application behind HTTPS.

Platform	How
Any container platform (Azure Container Apps, AWS ECS or App Runner, Kubernetes, a VM with Docker)	Build the image from the `Dockerfile`. Mount a persistent volume at `/app/data`. Set configuration through environment variables or secrets.
Azure App Service (Windows or Linux)	Publish the application, enable a managed identity, use `Auth:ManagedIdentity` for Azure connections
IIS on Windows Server	Install the ASP.NET Core 8 Hosting Bundle, host the published folder with an HTTPS binding, give the application pool identity access to the data folder
Linux server	systemd service (section 3.2) behind nginx, Caddy or Apache with HTTPS

In every case:

1. Terminate HTTPS at the load balancer or reverse proxy, and set `DocumentOrganizer__BehindReverseProxy=true` so client addresses in the audit log and secure cookies work correctly.
2. Point health checks at `GET /healthz`. It returns `200 {"status":"ok"}` without signing in.
3. Run **one instance** per data volume. Jobs run inside the application and state is in SQLite, so the application is not designed for several instances behind a load balancer.
4. Keep the data folder on fast, persistent storage that only the application can read.
5. Set `EnableDemoData=false` and, if local folders are used, `AllowedLocalRoots`.
6. Prefer role-based cloud access: managed identity on Azure, an IAM role on AWS.
7. Set `DOCORG_ADMIN_PASSWORD` once for the first start, then remove it.

8 Backup, retention and logs

What	Where	How to back up
Database	`data/document-organizer.db` (and its `-wal` file)	Stop the application, then copy the files; or snapshot the volume
Uploaded mapping files	`data/jobs/<Job ID>/`	Copy the folder
Reports	Downloaded by users	Store with the client's ticket or case
Application log	Console output (container log, systemd journal)	Collect with your normal log tooling

The database holds every job, item and audit event. It does not hold any document content or storage secrets.

9 Upgrading

1. Stop the application.
2. Back up the `data` folder.
3. Pull the new version (`git pull`) or replace the application files, keeping the `data` folder and your `appsettings.Production.json`. With Docker, rebuild or pull the new image; the volume keeps the data.
4. Start the application. The database is upgraded automatically.
5. Read the Release Notes (DO-RN-07) for configuration changes.

Upgrading from version 2.0 to 2.1

No configuration changes. New database columns are added automatically at start-up; existing jobs keep working with their single mapping file. File name cells listing several files are now split (one entry per file) and `NULL`-style placeholders are treated as empty, so scanning an old job again can change its counts. See the Release Notes.

Upgrading from version 1.0

Version 2.0 removes the SignUpID field and renames the traceability metadata keys on copied files (see the Release Notes, section 5). Existing jobs keep working. Add `"Provider"` to each existing storage connection for clarity (it is inferred if left out).

10 Running the automated tests

The end-to-end test runs the full flow against a running application:

```
node tests/e2e.mjs http://localhost:5080 <admin-password> all
```

`node tests/e2e-mapping.mjs http://localhost:5080 <admin-password>` checks mapping-file handling: several files per job, several file names in one cell, placeholders and repeated records.

For `tests/e2e.mjs`, `local` tests only folders; `cloud` runs a job for every reachable demo connection, each cloud source to a local folder, and Azure to S3 and back; `all` does both. The GitHub Actions workflow in `.github/workflows/ci.yml` builds the application and runs these tests against Azurite and moto on every push.

11 Go-live checklist

#	Check	Done
1	Application runs on the target machine or platform, and `/healthz` returns `ok`	☐
2	Admin password changed to a strong password; `DOCORG_ADMIN_PASSWORD` removed	☐
3	`EnableDemoData` set to `false`; emulator connections removed	☐
4	Storage connections added, secrets outside Git, and **Check access** passes for each source and destination	☐
5	Source access is read-only; destination access is limited to one container or bucket	☐
6	Operators and viewers created with the right roles	☐
7	UAT test cases (DO-UAT-06) passed on this installation	☐
8	Pilot job on a small test container or bucket reconciled with zero unexplained exceptions	☐
9	Backup of the `data` folder scheduled or agreed	☐
10	Shared hosting only: HTTPS, `BehindReverseProxy`, `AllowedLocalRoots`, single instance	☐